

数算 Cheat Sheet (Data Structures and Algorithms)

🎯 易错点与防坑指南

- 1. **不要忘记调用**：写完的函数一定要记得在自己写的程序中调用！
- 2. **边界特判**：特判起点终点是否相同、容器是否为空、`n=0` 或 `n=1`。
- 3. **RE (Runtime Error) 警报**：可能原因：数组越界、除以 0、递归爆栈。
- 4. **类型混淆**：将数组错用为数。

📖 常用算法 Tag 中英对照表

英文 Tag	中文含义	英文 Tag	中文含义
brute force	暴力枚举	Trie	字典树
implementation	模拟	DSU	并查集
greedy	贪心	segment tree	线段树
recursion	递归	lazy tag	懒惰标记
backtracking	回溯	Fenwick/BIT	树状数组
binary search	二分查找	dfs and similar	图中深搜
two pointers	双指针	topological order	拓扑排序
sliding window	滑动窗口	SCC	强连通分量
merge sort	归并排序	MST	最小生成树
bit manipulation	位运算	LCA	最近公共祖先
bitmask	状态压缩	binary lifting	倍增法
discretization	离散化	number theory	数论
linked list	链表	combinatorics	组合数学
binary tree	二叉树	geometry	计算几何
hash table	哈希表	Warnsdorff	骑士巡逻启发式规则
monotonic stack	单调栈	AST	抽象语法树

🔧 Python 基础与提速技巧

1. 快速输入与输出

```
import sys
import math
data = iter(sys.stdin.read().split())
```

```
n, m = int(next(data)), int(next(data))

x, y, z = sorted(map(int, input().split()))

while True:
    try:
        line = input()
        #处理每一行的逻辑
    except EOFError:
        break
```

输出格式化:

```
print("Hello", "World", sep="-", end="!")

name = "Alice"
age = 20
print(f"{name} is {age} years old.")

pi = 3.1415926
print(f"圆周率大约是: {pi:.2f}")

math.floor(3.14) # 3
math.floor(-3.14) # -4
math.ceil(3.14) # 4
math.ceil(-3.14) # -3
math.sqrt(4)
```

2. 时间优化与防爆栈

```
import sys
from functools import lru_cache

def solve():
    pass

if __name__ == '__main__':
    solve()

a if a < b else b #min()

sys.setrecursionlimit(300000)

@lru_cache(maxsize=None)
def avl(n):
    pass
```

3. 类型互化与 OOP

```
# 字符串与数互化
a = 6
b = str(a)
a == int(b)
s = 'b'
ord(s) - ord('a') == 1

# OOP
class Fraction:
    def __init__(self, a, b):
        pass
    def __str__(self):
        return f"{self.num}/{self.den}"
    def __add__(self, other):
        pass
```

数学与位运算 (Bit Manipulation)

包含：异或 $\wedge$ 、与 $\&$ 、或 $|$ 、取反 $\sim$ 、左移 $\ll$ 、右移 $\gg$ 。

```
n = 37
# 100101
n.bit_count() == 3
n.bit_length() == 6
s = bin(n)[2:] # 字符串100101

binary_str = "1011"
integer_value = int(binary_str, 2)
print(integer_value) # 输出: 11

# 颠倒二进制位
res = 0
for i in range(32):
    res = (res << 1) | (n & 1)
    n >>= 1

# 反码
mask = (1 << n.bit_length()) - 1
return mask ^ n

# 是否是2的幂次
i & (i - 1) == 0

# low_bit
low_bit = x & (-x)
# x = 12 = 1100 low_bit = 4
```

排序与二分查找

1. 基础排序

```
num.sort()
num.sort(key = lambda x: x.val, reverse = True)
num.sort(key = len)
```

2. 二分查找 (Python 自带)

```
import bisect
nums = [1, 2, 4, 4, 4, 6, 8]
# 索引: 0 1 2 3 4 5 6
idx_left = bisect.bisect_left(nums, 4)
idx_right = bisect.bisect_right(nums, 4)
print(f"bisect_left 插入位置: {idx_left}") # 输出: 2 (插在第一个 4 的前面)
print(f"bisect_right 插入位置: {idx_right}") # 输出: 5 (插在最后一个 4 的后面, 也就是 6 的前面)

num_4 = bisect.bisect_right(nums, 4) - bisect.bisect_left(nums, 4) # 4有3个

# 如果寻找列表中不存在的数字, 比如 5
idx_not_exist = bisect.bisect_left(nums, 5)
print(f"寻找 5 的位置: {idx_not_exist}") # 输出: 5 (插在最后一个 4 和 6 之间)

nums = [1, 2, 3, 4, 5, 6, 7]
idx = bisect.bisect_left(nums, 4, lo=2, hi=6) # 只在索引 2 到 5 的范围内寻找 4
```

3. 二分查找 (手写)

```
# 寻找第一个使key值>x的元素, 返回a中下标。若无返回len(a)
def bisect_strict_first(a, key, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if key(a[mid]) > x:
            hi = mid
        else:
            lo = mid + 1
    return lo

# 寻找第一个使key值>=x的元素, 返回a中下标。若无返回len(a)
def bisect_nonstrict_first(a, key, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if key(a[mid]) < x:
            lo = mid + 1
```

```
        else:
            hi = mid
    return lo
```

4. 排序算法与逆序对

```
# 逆序对模板(归并排序)
def count_inv_merge(a):
    if len(a) <= 1: return a, 0
    mid = len(a) // 2
    L, inv1 = count_inv_merge(a[:mid])
    R, inv2 = count_inv_merge(a[mid:])
    res, inv, i, j = [], inv1 + inv2, 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: res.append(L[i]); i += 1
        else: res.append(R[j]); inv += len(L) - i; j += 1
    return res + L[i:] + R[j:], inv

# arr = [5, 2, 6, 1]
# _, ans = count_inv_merge(arr)
# ans 即为结果 4

# 归并排序
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    res = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]: res.append(left[i]); i += 1
        else: res.append(right[j]); j += 1
    res.extend(left[i:])
    res.extend(right[j:])
    return res

# 快速排序
def quick_sort(arr, low, high):
    if low < high:
        pivot_idx = partition(arr, low, high)
        quick_sort(arr, low, pivot_idx - 1)
        quick_sort(arr, pivot_idx + 1, high)

def partition(arr, low, high):
    pivot = arr[low]
    left = low + 1
    right = high
    while True:
        while left <= right and arr[left] <= pivot: left += 1
        while left <= right and arr[right] >= pivot: right -= 1
```

```

        if left <= right:
            arr[left], arr[right] = arr[right], arr[left]
        else:
            break
    arr[low], arr[right] = arr[right], arr[low]
    return right

# 希尔排序
def shell_sort(arr):
    n = len(arr)
    gap = n // 2 # 初始化步长
    while gap > 0:
        for i in range(gap, n):
            temp = arr[i]
            j = i
            while j >= gap and arr[j - gap] > temp:
                arr[j] = arr[j - gap]
                j -= gap
            arr[j] = temp
        gap //= 2

```

基础数据结构与 Python 容器

1. Collections

```

# deque
from collections import deque
dq = deque()
dq.pop()
dq.append(1)
dq.appendleft(2)
dq.popleft()
dq.rotate(1) # 向右移动1步
dq.reverse()
history = deque(maxlen = 3) # 滑动窗口

# 查询时不报错的字典
import collections
count = collections.defaultdict(int)
count = collections.defaultdict(list)
class Node:
    def __init__(self, val):
        self.val = val
count = collections.defaultdict(lambda: Node(0))

# 计数函数 Counter
from collections import Counter
c1 = Counter("abracadabra")
c2 = Counter(['apple', 'banana', 'apple', 'orange'])
c3 = Counter(a=3, b=1)

```

```

c = Counter("abracadabra")
print(c.most_common(2)) # 输出: [('a', 5), ('b', 2)]

# LRU Cache
from collections import OrderedDict
class LRU Cache:
    def __init__(self, capacity: int):
        self.cache = OrderedDict()
        self.capacity = capacity
    def get(self, key: int) -> int:
        if key not in self.cache: return -1
        self.cache.move_to_end(key)
        return self.cache[key]
    def put(self, key: int, value: int) -> None:
        if key in self.cache: self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity: self.cache.popitem(last=False)

```

2. 栈与单调结构

```

# 调度场算法 (中缀转后序 RPN)
def shunting_yard(expr):
    prec = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}
    out, ops = [], []
    for t in expr.split():
        if t.isalnum(): out.append(t)
        elif t == '(': ops.append(t)
        elif t == ')':
            while ops[-1] != '(': out.append(ops.pop())
            ops.pop()
        else:
            while ops and ops[-1] != '(' and (prec[ops[-1]] > prec[t] or
            (prec[ops[-1]] == prec[t] and t != '^')):
                out.append(ops.pop())
            ops.append(t)
    return out + ops[::-1]
# expression = "3 + 4 * 2 / ( 1 - 5 ) ^ 2"
# 输出: 3 4 2 * 1 5 - 2 ^ / +

# 单调栈 (找右侧第一个比他大的元素的索引)
def next_greater_element(nums):
    ans = [-1] * len(nums)
    stack = []
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x: ans[stack.pop()] = i
        stack.append(i)
    return ans

# 单调队列 (找给定范围内最大值的索引)
from collections import deque
def max_sliding_window(nums, k):

```

```

q = deque()
ans = []
for i, x in enumerate(nums):
    while q and nums[q[-1]] <= x: q.pop()
    q.append(i)
    if q[0] <= i - k: q.popleft()
    ans.append(q[0])
return ans

# 完美交易窗口 (单调栈应用)
import sys
from bisect import bisect_right
raw_data = sys.stdin.read().split()
if raw_data:
    ans = 0
    n = int(raw_data[0])
    data = [int(raw_data[i]) for i in range(1, n + 1)]
    decrease_stack = list()
    pre_min = list()
    for i in range(0, n):
        while decrease_stack and data[decrease_stack[-1]] < data[i]:
            decrease_stack.pop()
        if decrease_stack: pre_min.append(decrease_stack[-1])
        else: pre_min.append(-1)
        decrease_stack.append(i)
    increase_stack = list()
    for i in range(0, n):
        while increase_stack and data[increase_stack[-1]] >= data[i]:
            increase_stack.pop()
        target = pre_min[i]
        key = bisect_right(increase_stack, target)
        if key < len(increase_stack): ans = max(ans, i - increase_stack[key] + 1)
        increase_stack.append(i)
    print(ans)

```

3. 堆 (Priority Queue)

```

# 哈夫曼编码 (每次将最小的两个节点相连)
import heapq
from collections import Counter
def huffman_min(text):
    if not text: return {}
    h = [[freq, [char, ""]] for char, freq in Counter(text).items()]
    heapq.heapify(h)
    while len(h) > 1:
        lo, hi = heapq.heappop(h), heapq.heappop(h)
        for pair in lo[1:]: pair[1] = '0' + pair[1]
        for pair in hi[1:]: pair[1] = '1' + pair[1]
        heapq.heappush(h, [lo[0] + hi[0], [lo[1:] + hi[1:]]])
    return dict(h[0][1:])

```



```
# Python自带堆 (heapq)
import heapq
nums = [5, 7, 9, 1, 3]
heapq.heapify(nums)
heapq.heappush(nums, 2)
min_val = heapq.heappop(nums)

# 手搓堆
class MinHeap:
    def __init__(self):
        self.heap = []
    def push(self, val):
        self.heap.append(val)
        self._sift_up(len(self.heap) - 1)
    def pop(self):
        if not self.heap: return None
        if len(self.heap) == 1: return self.heap.pop()
        root = self.heap[0]
        self.heap[0] = self.heap.pop()
        self._sift_down(0)
        return root
    def _sift_up(self, idx):
        while idx > 0:
            parent = (idx - 1) // 2
            if self.heap[idx] < self.heap[parent]:
                self.heap[idx], self.heap[parent] = self.heap[parent],
self.heap[idx]
                idx = parent
            else: break
    def _sift_down(self, idx):
        n = len(self.heap)
        while 2 * idx + 1 < n:
            left = 2 * idx + 1
            right = 2 * idx + 2
            smallest = left
            if right < n and self.heap[right] < self.heap[left]: smallest = right
            if self.heap[idx] > self.heap[smallest]:
                self.heap[idx], self.heap[smallest] = self.heap[smallest],
self.heap[idx]
                idx = smallest
            else: break

# Floyd 建堆
def build_heap(arr):
    n = len(arr)
    def sift_down(idx):
        while 2 * idx + 1 < n:
            child = 2 * idx + 1
            if child + 1 < n and arr[child + 1] < arr[child]: child += 1
            if arr[idx] > arr[child]:
                arr[idx], arr[child] = arr[child], arr[idx]
                idx = child
            else: break
    for i in range(n // 2 - 1, -1, -1):
```

```

sift_down(i)

# 维护中位数
from heapq import heappush, heappop
class MedianFinder:
    def __init__(self):
        self.small = []
        self.large = []
    def addNum(self, num: int) -> None:
        heappush(self.small, -num)
        heappush(self.large, -heappop(self.small))
        if len(self.small) < len(self.large):
            heappush(self.small, -heappop(self.large))
    def findMedian(self) -> float:
        if len(self.small) > len(self.large): return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2.0

```

树与森林

1. 基础理论

- **完全二叉树 (complete binary tree)**: 只有最下面两层节点的度数可以小于 2, 且最下面一层的节点都集中在该层最左边的连续位置上。
- **完美二叉树 (perfect binary tree)**: 所有叶节点的深度均相同, 且所有非叶节点的子节点数量均为 2 的二叉树。
- **m-叉树**: 总分支数等于总节点数-1。
- **非空二叉树公式**: 叶节点数 n_0 , 度数为 2 (两个子节点) $n_2: n_0 = n_2 + 1$
- **卡特兰数**: n 个数的合法出栈序列数和 n 个节点构造的不同二叉树的数目。

$$C_n = \frac{(2n)!}{(n+1)!n!}$$

(1, 1, 2, 5, 14, 42, 132, 4429, 1430)

2. 二叉树的遍历与构建

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val; self.left = left; self.right = right

# 中序遍历 (栈)
def inorder_traversal(root):
    ans, stack, cur = [], [], root
    while stack or cur:
        while cur:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        ans.append(cur.val)

```

```

        cur = cur.right
    return ans

# 中序遍历 (Morris)
def morris_inorder(root):
    ans, cur = [], root
    while cur:
        if cur.left is None:
            ans.append(cur.val); cur = cur.right
        else:
            most_right = cur.left
            while most_right.right and most_right.right != cur: most_right = most_right.right
            if most_right.right is None:
                most_right.right = cur; cur = cur.left
            else:
                most_right.right = None
                ans.append(cur.val); cur = cur.right
    return ans

# 前序、中序构建树
def buildTree_simple(preorder: list[int], inorder: list[int]) -> TreeNode:
    if not preorder or not inorder: return None
    root = TreeNode(preorder[0])
    mid = inorder.index(preorder[0])
    root.left = buildTree_simple(preorder[1:mid+1], inorder[:mid])
    root.right = buildTree_simple(preorder[mid+1:], inorder[mid+1:])
    return root

# 前序遍历构建二叉搜索树 (BST)
from collections import deque
import math
def bstFromPreorder(preorder: list[int]) -> TreeNode:
    q = deque(preorder)
    def build(upper_bound):
        if not q or q[0] >= upper_bound: return None
        root_val = q.popleft()
        root = TreeNode(root_val)
        root.left = build(root_val)
        root.right = build(upper_bound)
        return root
    return build(math.inf)

```

3. 特殊树结构与操作

```

# 笛卡尔树 (Cartesian Tree)
def buildCartesianTree(nums: list[int]) -> TreeNode:
    if not nums: return None
    stack = []
    for num in nums:
        node = TreeNode(num)

```

```

        last_popped = None
        while stack and stack[-1].val > num: last_popped = stack.pop()
        node.left = last_popped
        if stack: stack[-1].right = node
        stack.append(node)
    return stack[0]

# 二叉树与森林互化 (左孩子右兄弟)
class BinaryNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val; self.left = left; self.right = right
class TreeNodeMulti:
    def __init__(self, val=0):
        self.val = val; self.children = []

def binaryTreeToForest(root: BinaryNode) -> list[TreeNodeMulti]:
    if not root: return []
    forest, curr = [], root
    while curr:
        tree_root = TreeNodeMulti(curr.val)
        tree_root.children = binaryTreeToForest(curr.left)
        forest.append(tree_root)
        curr = curr.right
    return forest

def forestToBinaryTree(forest: list[TreeNodeMulti]) -> BinaryNode:
    if not forest: return None
    head = curr = None
    for tree_node in forest:
        bin_node = BinaryNode(tree_node.val)
        bin_node.left = forestToBinaryTree(tree_node.children)
        if not head: head = bin_node
        else: curr.right = bin_node
        curr = bin_node
    return head

```

4. 欧拉序列与 LCA

欧拉序列长度为 $2n-1$ 。LCA 极值定理：一个节点在序列中出现的总次数，严格等于它的出度加一。

```

import sys
sys.setrecursionlimit(200000)
class EulerTourNaiveLCA:
    def __init__(self, n: int, edges: list[tuple[int, int]], root: int = 1):
        self.n = n
        self.adj = [[] for _ in range(n + 1)]
        for u, v in edges:
            self.adj[u].append(v); self.adj[v].append(u)
        self.euler, self.depths, self.first = [], [], [0] * (n + 1)
        self._dfs(root, 0, 1)
    def _dfs(self, u: int, p: int, d: int) -> None:

```

```

        self.first[u] = len(self.euler)
        self.euler.append(u); self.depths.append(d)
        for v in self.adj[u]:
            if v != p:
                self._dfs(v, u, d + 1)
                self.euler.append(u); self.depths.append(d)
def query_lca(self, u: int, v: int) -> int:
    left, right = self.first[u], self.first[v]
    if left > right: left, right = right, left
    min_depth, lca_node = float('inf'), -1
    for i in range(left, right + 1):
        if self.depths[i] < min_depth:
            min_depth = self.depths[i]
            lca_node = self.euler[i]
    return lca_node

```

5. 树上 DP 与路径和

```

# 树上路径总和
import collections
class SolutionPathSum:
    def pathSum(self, root: TreeNode, targetSum: int) -> int:
        prefix = collections.defaultdict(int)
        prefix[0] = 1
        def dfs(root, curr):
            if not root: return 0
            ret = 0
            curr += root.val
            ret += prefix[curr - targetSum]
            prefix[curr] += 1
            ret += dfs(root.left, curr) + dfs(root.right, curr)
            prefix[curr] -= 1
            return ret
        return dfs(root, 0)

# 二叉树中最大路径和
class SolutionMaxPath:
    def __init__(self):
        self.maxSum = float("-inf")
    def maxPathSum(self, root: TreeNode) -> int:
        def maxGain(node):
            if not node: return 0
            leftGain = max(maxGain(node.left), 0)
            rightGain = max(maxGain(node.right), 0)
            priceNewpath = node.val + leftGain + rightGain
            self.maxSum = max(self.maxSum, priceNewpath)
            return node.val + max(leftGain, rightGain)
        maxGain(root)
        return self.maxSum

# 树上 DP (左孩子右兄弟转化法/选课)

```

```

import sys
sys.setrecursionlimit(20000)
def solve_course_selection(n: int, m: int, values: list[int], dependencies:
list[int]) -> int:
    children = [[] for _ in range(n + 1)]
    for i in range(1, n + 1): children[dependencies[i]].append(i)
    left, right = [-1] * (n + 1), [-1] * (n + 1)
    for u in range(n + 1):
        if children[u]:
            left[u] = children[u][0]
            for k in range(len(children[u]) - 1): right[children[u][k]] =
children[u][k + 1]
    dp = [[-1] * (m + 2) for _ in range(n + 1)]
    def dfs(u: int, j: int) -> int:
        if j == 0: return 0
        if u == -1: return -float('inf')
        if dp[u][j] != -1: return dp[u][j]
        res_not_pick = -float('inf') if u == 0 else dfs(right[u], j)
        res_pick = -float('inf')
        for k in range(j):
            val = dfs(left[u], k) + dfs(right[u], j - 1 - k) + values[u]
            if val > res_pick: res_pick = val
        dp[u][j] = max(res_not_pick, res_pick)
        return dp[u][j]
    return dfs(0, m + 1)

# 树上 DP (背包)
dp = [[0 for i in range(0, m + 1)] for j in range(0, t + 1)]
def dfs_dp(u):
    for i in range(weight_scc[u], m + 1): dp[u][i] = val_scc[u]
    for v in graph_scc[u]:
        dfs_dp(v)
        for j in range(m, weight_scc[u] - 1, -1):
            for k in range(0, j - weight_scc[u] + 1):
                dp[u][j] = max(dp[u][j], dp[u][j - k] + dp[v][k])
dfs_dp(0)
print(dp[0][m])

```

高级数据结构

1. 并查集 (DSU)

```

# 基础并查集
parent = list(range(n + 1)); size = [1] * (n + 1)
def find(x):
    if parent[x] != x: parent[x] = find(parent[x])
    return parent[x]
def union(x, y):
    root_x, root_y = find(x), find(y)
    if root_x == root_y: return False

```

```

    if size[root_x] < size[root_y]: root_x, root_y = root_y, root_x
    parent[root_y] = root_x
    size[root_x] += size[root_y]
    return True

# 带权并查集
def root(x):
    if father[x] != x:
        old_father = father[x]
        father[x] = root(father[x])
        num[x] += num[old_father]
        return father[x]
    else: return x
def build(a, b, d):
    if length[a] < length[b]: a, b, d = b, a, -d
    father[b] = a
    length[a] += length[b]
    num[b] = d

# 种类并查集
def solve_food_chain(n, m, queries):
    parent = list(range(3 * n + 1))
    def find(x):
        if parent[x] != x: parent[x] = find(parent[x])
        return parent[x]
    def union(x, y):
        root_x, root_y = find(x), find(y)
        if root_x != root_y: parent[root_x] = root_y
    fake_count = 0
    for type, x, y in queries:
        if x > n or y > n or (type == 2 and x == y):
            fake_count += 1; continue
        if type == 1:
            if find(x) == find(y + n) or find(x) == find(y + 2 * n): fake_count +=
1
            else:
                union(x, y); union(x + n, y + n); union(x + 2 * n, y + 2 * n)
        else:
            if find(x) == find(y) or find(x) == find(y + 2 * n): fake_count += 1
            else:
                union(x + n, y); union(x, y + 2 * n); union(x + 2 * n, y + n)
    return fake_count

```

2. 字典树 (Trie)

```

# 字典建树
class Trie:
    def __init__(self):
        self.root = {}
    def insert(self, word: str) -> None:
        node = self.root

```

```

    for char in word: node = node.setdefault(char, {})
    node['#'] = True
def search(self, word: str) -> bool:
    node = self.root
    for char in word:
        if char not in node: return False
        node = node[char]
    return '#' in node
def startsWith(self, prefix: str) -> bool:
    node = self.root
    for char in prefix:
        if char not in node: return False
        node = node[char]
    return True
def delete(self, word: str) -> None:
def _dfs(node: dict, i: int) -> bool:
    if i == len(word):
        if '#' in node:
            del node['#']
            return len(node) == 0
        return False
    char = word[i]
    if char not in node: return False
    should_prune_child = _dfs(node[char], i + 1)
    if should_prune_child:
        del node[char]
        return len(node) == 0 and '#' not in node
    return False
_dfs(self.root, 0)

```

数组模拟字典树

```

class TrieStatic:
    def __init__(self, max_nodes: int = 100005):
        self.ch = [[0] * 26 for _ in range(max_nodes)]
        self.cnt = [0] * max_nodes
        self.idx = 0
    def insert(self, word: str) -> None:
        p = 0
        for char in word:
            u = ord(char) - ord('a')
            if not self.ch[p][u]:
                self.idx += 1
                self.ch[p][u] = self.idx
            p = self.ch[p][u]
        self.cnt[p] += 1
    def search(self, word: str) -> int:
        p = 0
        for char in word:
            u = ord(char) - ord('a')
            if not self.ch[p][u]: return 0
            p = self.ch[p][u]
        return self.cnt[p]
    def delete(self, word: str) -> None:
        if self.search(word) == 0: return

```



```

p = 0
for char in word:
    u = ord(char) - ord('a')
    p = self.ch[p][u]
self.cnt[p] -= 1

```

3. 树状数组 (BIT) 与 线段树 (Segment Tree)

```

# 逆序对模板 (树状数组)
def count_inv_bit(a):
    rank = {v: i for i, v in enumerate(sorted(set(a)), 1)}
    tree, inv = [0] * (len(rank) + 1), 0
    for x in a[::-1]:
        i, j = rank[x] - 1, rank[x]
        while i: inv += tree[i]; i -= i & -i
        while j < len(tree): tree[j] += 1; j += j & -j
    return inv

# 线段树 (迭代)
class SegmentTreeIterative:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (2 * self.n)
        for i in range(self.n): self.tree[self.n + i] = data[i]
        for i in range(self.n - 1, 0, -1): self.tree[i] = self.tree[i << 1] +
self.tree[i << 1 | 1]
    def update(self, pos, value):
        pos += self.n
        self.tree[pos] = value
        while pos > 1:
            pos >>= 1
            self.tree[pos] = self.tree[pos << 1] + self.tree[pos << 1 | 1]
    def query(self, l, r):
        l += self.n; r += self.n; res = 0
        while l <= r:
            if l % 2 == 1: res += self.tree[l]; l += 1
            if r % 2 == 0: res += self.tree[r]; r -= 1
            l >>= 1; r >>= 1
        return res

# 线段树 (递归+lazy mark)
class SegmentTreeLazy:
    def __init__(self, data):
        self.n = len(data)
        self.data = [0] + data
        self.tree = [0] * (4 * self.n + 1)
        self.lazy = [0] * (4 * self.n + 1)
        self._build(1, 1, self.n)
    def _build(self, p, l, r):
        if l == r:
            self.tree[p] = self.data[l]

```

```

        return
    mid = (l + r) // 2
    self._build(p << 1, l, mid)
    self._build(p << 1 | 1, mid + 1, r)
    self.tree[p] = self.tree[p << 1] + self.tree[p << 1 | 1]
def _push_down(self, p, l_len, r_len):
    if self.lazy[p]:
        lz = self.lazy[p]
        self.lazy[p << 1] += lz; self.tree[p << 1] += lz * l_len
        self.lazy[p << 1 | 1] += lz; self.tree[p << 1 | 1] += lz * r_len
        self.lazy[p] = 0
def update_range(self, p, l, r, ql, qr, val):
    if ql <= l and r <= qr:
        self.tree[p] += val * (r - l + 1); self.lazy[p] += val; return
    mid = (l + r) // 2
    self._push_down(p, mid - l + 1, r - mid)
    if ql <= mid: self.update_range(p << 1, l, mid, ql, qr, val)
    if qr > mid: self.update_range(p << 1 | 1, mid + 1, r, ql, qr, val)
    self.tree[p] = self.tree[p << 1] + self.tree[p << 1 | 1]
def query(self, p, l, r, ql, qr):
    if ql <= l and r <= qr: return self.tree[p]
    mid = (l + r) // 2
    self._push_down(p, mid - l + 1, r - mid)
    res = 0
    if ql <= mid: res += self.query(p << 1, l, mid, ql, qr)
    if qr > mid: res += self.query(p << 1 | 1, mid + 1, r, ql, qr)
    return res
def update(self, ql, qr, val): self.update_range(1, 1, self.n, ql, qr, val)
def query_sum(self, ql, qr): return self.query(1, 1, self.n, ql, qr)

```

排队 (线段树应用)

```

import sys
import bisect
import collections
raw_data = sys.stdin.read().split()
if raw_data:
    n, d = int(raw_data[0]), int(raw_data[1])
    height = [int(raw_data[i]) for i in range(2, 2 + n)]
    class segment_tree:
        def __init__(self, nums):
            self.n = len(nums)
            self.nums = [0] * self.n; self.tree = [0] * (4 * self.n)
        def _build(self, node, start, end, index, val):
            if start == end:
                self.tree[node] = val; self.nums[start] = val; return
            left, right, mid = node * 2 + 1, node * 2 + 2, (start + end) // 2
            if mid >= index: self._build(left, start, mid, index, val)
            else: self._build(right, mid + 1, end, index, val)
            self.tree[node] = max(self.tree[left], self.tree[right])
        def _search(self, node, start, end, index_left, index_right):
            if index_left > index_right: return 0
            if index_left <= start and index_right >= end: return self.tree[node]
            seq, left, right, mid = 0, node * 2 + 1, node * 2 + 2, (start + end)

```

// 2

```

        if mid >= index_left: seq = max(seq, self._search(left, start, mid,
index_left, index_right))
        if mid < index_right: seq = max(seq, self._search(right, mid + 1, end,
index_left, index_right))
        return seq
    def add(self, index, val): self._build(0, 0, self.n - 1, index, val)
    def find(self, index_left, index_right): return self._search(0, 0, self.n
- 1, index_left, index_right)
    nums = [[height[i], i] for i in range(0, n)]
    tree = segment_tree(nums)
    nums.sort()
    match = dict(); ans = collections.defaultdict(list)
    for i in range(0, n): match[nums[i][1]] = i
    for i in range(0, n):
        key = match[i]
        tall = nums[key][0]
        left = bisect.bisect_right(nums, [tall - d - 1, float('inf')]) - 1
        right = bisect.bisect_left(nums, [tall + d + 1, -float('inf')])
        temp = max(tree.find(0, left), tree.find(right, n - 1))
        tree.add(key, temp + 1)
    for i in range(0, n): ans[tree.nums[i]].append(nums[i][0])
    for x in ans.values(): x.sort()
    final = list(ans.items())
    final.sort()
    for x in final: print(*x[1], end = ' ')

```

4. 高级树结构

```

# KD-tree
import math
class KNode:
    def __init__(self, point, axis, left=None, right=None):
        self.point = point; self.axis = axis; self.left = left; self.right = right
class KDTree:
    def __init__(self, points):
        self.k = len(points[0]) if points else 0
        self.root = self._build(points, depth=0)
    def _build(self, points, depth):
        if not points: return None
        axis = depth % self.k
        points.sort(key=lambda x: x[axis])
        mid = len(points) // 2
        return KNode(
            point=points[mid], axis=axis,
            left=self._build(points[:mid], depth + 1),
            right=self._build(points[mid + 1:], depth + 1)
        )
    def nearest_neighbor(self, target):
        best_point, best_dist = None, float('inf')
        def dist_sq(p1, p2): return sum((a - b) ** 2 for a, b in zip(p1, p2))
        def _search(node):

```

```

        nonlocal best_point, best_dist
        if not node: return
        d = dist_sq(node.point, target)
        if d < best_dist:
            best_dist = d; best_point = node.point
        axis = node.axis
        diff = target[axis] - node.point[axis]
        good_side = node.left if diff < 0 else node.right
        bad_side = node.right if diff < 0 else node.left
        _search(good_side)
        if diff ** 2 < best_dist: _search(bad_side)
    _search(self.root)
    return best_point, math.sqrt(best_dist)

# AVL树
class AVLNode:
    def __init__(self, key):
        self.key = key; self.left = None; self.right = None; self.height = 1
class AVLTree:
    def get_height(self, node):
        if not node: return 0
        return node.height
    def get_balance(self, node):
        if not node: return 0
        return self.get_height(node.left) - self.get_height(node.right)
    def right_rotate(self, y):
        x = y.left
        T2 = x.right
        x.right = y; y.left = T2
        y.height = max(self.get_height(y.left), self.get_height(y.right)) + 1
        x.height = max(self.get_height(x.left), self.get_height(x.right)) + 1
        return x
    def left_rotate(self, x):
        y = x.right
        T2 = y.left
        y.left = x; x.right = T2
        x.height = max(self.get_height(x.left), self.get_height(x.right)) + 1
        y.height = max(self.get_height(y.left), self.get_height(y.right)) + 1
        return y
    def insert(self, root, key):
        if not root: return AVLNode(key)
        elif key < root.key: root.left = self.insert(root.left, key)
        else: root.right = self.insert(root.right, key)
        root.height = 1 + max(self.get_height(root.left),
self.get_height(root.right))
        balance = self.get_balance(root)
        if balance > 1 and key < root.left.key: return self.right_rotate(root)
        if balance < -1 and key > root.right.key: return self.left_rotate(root)
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)

```

```
        return self.left_rotate(root)
    return root
```

图论算法

1. 遍历与环检测

```
# 三色标记法 bfs
from collections import deque
def bfs_tricolor(graph: dict, start: int):
    color = {u: 0 for u in graph}; color[start] = 1
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            if color[v] == 0: color[v] = 1; queue.append(v)
        color[u] = 2

# 三色标记法 dfs (带时间戳)
def dfs_tricolor_timestamps(graph: dict):
    color, d, f, time = {u: 0 for u in graph}, {}, {}, 0
    def dfs_visit(u):
        nonlocal time
        time += 1
        d[u] = time; color[u] = 1
        for v in graph[u]:
            if color[v] == 0: dfs_visit(v)
            elif color[v] == 1: pass
            elif color[v] == 2: pass
        time += 1
        f[u] = time; color[u] = 2
    for u in graph:
        if color[u] == 0: dfs_visit(u)
    return d, f, color

# 无向图判环 dfs
def has_cycle_undirected(num_nodes: int, edges: list[list[int]]) -> bool:
    graph = {i: [] for i in range(num_nodes)}
    for u, v in edges: graph[u].append(v); graph[v].append(u)
    visited = [False] * num_nodes
    def dfs(u, parent):
        visited[u] = True
        for v in graph[u]:
            if not visited[v]:
                if dfs(v, u): return True
            elif v != parent: return True
        return False
    for i in range(num_nodes):
        if not visited[i]:
            if dfs(i, -1): return True
```

```

    return False

# 无向图判环 并查集 (DSU)
def has_cycle_dsu(num_nodes: int, edges: list[list[int]]) -> bool:
    parent = list(range(num_nodes))
    def find(x):
        if parent[x] != x: parent[x] = find(parent[x])
        return parent[x]
    for u, v in edges:
        root_u, root_v = find(u), find(v)
        if root_u == root_v: return True
        parent[root_u] = root_v
    return False

```

2. 拓扑排序 (Topological Sort)

```

# 有向图判环/拓扑排序 Kahn(bfs)
from collections import deque
def kahn_topo_sort(num_nodes: int, edges: list[list[int]]) -> list[int]:
    graph = {i: [] for i in range(num_nodes)}
    in_degree = [0] * num_nodes
    for u, v in edges:
        graph[u].append(v); in_degree[v] += 1
    queue = deque([i for i in range(num_nodes) if in_degree[i] == 0])
    topo_order = []
    while queue:
        u = queue.popleft()
        topo_order.append(u)
        for v in graph[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0: queue.append(v)
    if len(topo_order) == num_nodes: return topo_order
    else: return []

# 有向图判环/拓扑排序 dfs
def dfs_topo_sort(num_nodes: int, edges: list[list[int]]) -> list[int]:
    graph = {i: [] for i in range(num_nodes)}
    for u, v in edges: graph[u].append(v)
    color, topo_order, has_cycle = [0] * num_nodes, [], False
    def dfs(u):
        nonlocal has_cycle
        if has_cycle: return
        color[u] = 1
        for v in graph[u]:
            if color[v] == 0: dfs(v)
            elif color[v] == 1:
                has_cycle = True; return
        color[u] = 2; topo_order.append(u)
    for i in range(num_nodes):
        if color[i] == 0: dfs(i)

```

```
if has_cycle: return []
return topo_order[::-1]
```

3. 强连通分量 (SCC) 与缩点

```
# 强连通元 SCC Tarjan
def tarjan_and_condense(num_nodes: int, edges: list[list[int]]):
    graph = {i: [] for i in range(num_nodes)}
    for u, v in edges: graph[u].append(v)
    dfn, low, in_stack = [-1] * num_nodes, [-1] * num_nodes, [False] * num_nodes
    stack, sccs, scc_id = [], [], [-1] * num_nodes
    timer = scc_count = 0
    def dfs(u):
        nonlocal timer, scc_count
        dfn[u] = low[u] = timer; timer += 1
        stack.append(u); in_stack[u] = True
        for v in graph[u]:
            if dfn[v] == -1:
                dfs(v); low[u] = min(low[u], low[v])
            elif in_stack[v]: low[u] = min(low[u], dfn[v])
        if dfn[u] == low[u]:
            scc = []
            while True:
                v = stack.pop(); in_stack[v] = False
                scc.append(v); scc_id[v] = scc_count
                if v == u: break
            sccs.append(scc); scc_count += 1
    for i in range(num_nodes):
        if dfn[i] == -1: dfs(i)
    dag = {i: set() for i in range(scc_count)}
    in_degree = [0] * scc_count
    for u, v in edges:
        u_id, v_id = scc_id[u], scc_id[v]
        if u_id != v_id and v_id not in dag[u_id]:
            dag[u_id].add(v_id); in_degree[v_id] += 1
    return sccs, scc_id, dag, in_degree

# 强连通元 SCC Kosaraju
def kosaraju_and_condense(num_nodes: int, edges: list[list[int]]):
    graph = {i: [] for i in range(num_nodes)}
    rev_graph = {i: [] for i in range(num_nodes)}
    for u, v in edges:
        graph[u].append(v); rev_graph[v].append(u)
    visited, order = [False] * num_nodes, []
    def dfs1(u):
        visited[u] = True
        for v in graph[u]:
            if not visited[v]: dfs1(v)
        order.append(u)
    for i in range(num_nodes):
        if not visited[i]: dfs1(i)
```

```

visited, sccs = [False] * num_nodes, []
def dfs2(u, current_scc):
    visited[u] = True; current_scc.append(u)
    for v in rev_graph[u]:
        if not visited[v]: dfs2(v, current_scc)
for i in reversed(order):
    if not visited[i]:
        scc = []
        dfs2(i, scc); sccs.append(scc)
num_scc = len(sccs)
scc_id = [0] * num_nodes
for i, component in enumerate(sccs):
    for node in component: scc_id[node] = i
dag = {i: set() for i in range(num_scc)}
in_degree = [0] * num_scc
for u, v in edges:
    u_id, v_id = scc_id[u], scc_id[v]
    if u_id != v_id:
        if v_id not in dag[u_id]:
            dag[u_id].add(v_id); in_degree[v_id] += 1
return sccs, scc_id, dag, in_degree

```

4. 最短路径

```

# 单源最短路 Dijkstra (无负权)
import heapq
def dijkstra(num_nodes: int, edges: list[list[int]], start: int) -> list[int]:
    graph = {i: [] for i in range(num_nodes)}
    for u, v, w in edges: graph[u].append((v, w))
    dist = [float('inf')] * num_nodes; dist[start] = 0
    pq = [(0, start)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue
        for v, weight in graph[u]:
            new_dist = d + weight
            if new_dist < dist[v]:
                dist[v] = new_dist
                heapq.heappush(pq, (new_dist, v))
    return dist

def dijkstra_with_path(num_nodes: int, edges: list[list[int]], start: int, target: int):
    graph = {i: [] for i in range(num_nodes)}
    for u, v, w in edges: graph[u].append((v, w))
    dist = [float('inf')] * num_nodes; dist[start] = 0
    parent = [-1] * num_nodes; pq = [(0, start)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue
        if u == target: break

```



```

        for v, weight in graph[u]:
            new_dist = d + weight
            if new_dist < dist[v]:
                dist[v] = new_dist; parent[v] = u
                heapq.heappush(pq, (new_dist, v))
    if dist[target] == float('inf'): return float('inf'), []
    path, curr = [], target
    while curr != -1:
        path.append(curr); curr = parent[curr]
    path.reverse()
    return dist[target], path

# 单源最短路 Bellman-Ford (有负权)
def bellman_ford(num_nodes: int, edges: list[list[int]], start: int):
    dist = [float('inf')] * num_nodes; dist[start] = 0
    for _ in range(num_nodes - 1):
        for u, v, weight in edges:
            if dist[u] != float('inf') and dist[u] + weight < dist[v]: dist[v] =
dist[u] + weight
    for u, v, weight in edges:
        if dist[u] != float('inf') and dist[u] + weight < dist[v]: return "负权
环!"
    return dist

# 单源最短路 Bellman-Ford (限制路径数 K)
def shortest_path_with_k_edges(num_nodes: int, edges: list[list[int]], start: int,
target: int, K: int):
    dist = [float('inf')] * num_nodes; dist[start] = 0
    for _ in range(K):
        backup = list(dist)
        for u, v, weight in edges:
            if backup[u] != float('inf') and backup[u] + weight < dist[v]: dist[v]
= backup[u] + weight
    return dist[target]

# 单源最短路 SPFA (Bellman-Ford) (SLF优化) 有负权
from collections import deque
def spfa_slf_with_path(n: int, edges: list[list[int]], start: int, target: int):
    g = [[] for _ in range(n)]
    for u, v, w in edges: g[u].append((v, w))
    dist = [float('inf')] * n; dist[start] = 0
    parent = [-1] * n; in_q = [False] * n; in_q[start] = True
    cnt = [0] * n; cnt[start] = 1
    q = deque([start])
    while q:
        u = q.popleft()
        in_q[u] = False
        for v, w in g[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w; parent[v] = u
                if not in_q[v]:
                    if q and dist[v] < dist[q[0]]: q.appendleft(v)
                    else: q.append(v)
                    in_q[v] = True; cnt[v] += 1

```

```

        if cnt[v] >= n: return float('inf'), []
    if dist[target] == float('inf'): return float('inf'), []
    path, curr = [], target
    while curr != -1: path.append(curr); curr = parent[curr]
    path.reverse()
    return dist[target], path

# 多源最短路 Floyd-Warshall
def floyd_warshall_with_path(n: int, edges: list[list[int]]):
    dist = [[float('inf')] * n for _ in range(n)]
    nxt = [[-1] * n for _ in range(n)]
    for i in range(n): dist[i][i] = 0; nxt[i][i] = i
    for u, v, w in edges:
        dist[u][v] = min(dist[u][v], w); nxt[u][v] = v
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != float('inf') and dist[k][j] != float('inf'):
                    if dist[i][k] + dist[k][j] < dist[i][j]:
                        dist[i][j] = dist[i][k] + dist[k][j]
                        nxt[i][j] = nxt[i][k]
    for i in range(n):
        if dist[i][i] < 0: return [], []
    return dist, nxt

def get_shortest_path(nxt: list[list[int]], start: int, end: int) -> list[int]:
    if nxt[start][end] == -1: return []
    path = [start]
    while start != end:
        start = nxt[start][end]; path.append(start)
    return path

# 带限制最短路
def constrained_shortest_path(n, edges, start, target, max_cost):
    graph = {i: [] for i in range(n)}
    for u, v, length, cost in edges: graph[u].append((v, length, cost))
    dist = [[float('inf')] * (max_cost + 1) for _ in range(n)]
    dist[start][0] = 0
    pq = [(0, start, 0)]
    while pq:
        d, u, c = heapq.heappop(pq)
        if d > dist[u][c]: continue
        for v, length, edge_cost in graph[u]:
            next_cost, next_dist = c + edge_cost, d + length
            if next_cost <= max_cost:
                if next_dist < dist[v][next_cost]:
                    dist[v][next_cost] = next_dist
                    heapq.heappush(pq, (next_dist, v, next_cost))
    min_final_distance = min(dist[target])
    return min_final_distance if min_final_distance != float('inf') else -1

```

5. 最小生成树 (MST)

```

# Prim 算法（节点扩张）（适用于稠密图）
import heapq
def prim_mst(num_nodes: int, edges: list[list[int]]) -> int:
    graph = {i: [] for i in range(num_nodes)}
    for u, v, w in edges:
        graph[u].append((v, w)); graph[v].append((u, w))
    visited, min_heap = [False] * num_nodes, [(0, 0)]
    total_cost = nodes_connected = 0
    while min_heap and nodes_connected < num_nodes:
        weight, u = heapq.heappop(min_heap)
        if visited[u]: continue
        visited[u] = True
        total_cost += weight; nodes_connected += 1
        for v, w in graph[u]:
            if not visited[v]: heapq.heappush(min_heap, (w, v))
    return total_cost if nodes_connected == num_nodes else -1

# Kruskal 算法（并查集）（适用于稀疏图）
def kruskal_mst(num_nodes: int, edges: list[list[int]]) -> int:
    edges.sort(key=lambda x: x[2])
    parent = list(range(num_nodes))
    def find(x):
        if parent[x] != x: parent[x] = find(parent[x])
        return parent[x]
    total_cost = edges_used = 0
    for u, v, weight in edges:
        root_u, root_v = find(u), find(v)
        if root_u != root_v:
            parent[root_u] = root_v
            total_cost += weight; edges_used += 1
            if edges_used == num_nodes - 1: break
    return total_cost if edges_used == num_nodes - 1 else -1

# 0-1 最小生成树（补图连通块）
from collections import deque
def zero_one_mst_complement(num_nodes: int, explicit_edges_cost_1:
list[list[int]]) -> int:
    adj_cost_1 = {i: set() for i in range(num_nodes)}
    for u, v in explicit_edges_cost_1:
        adj_cost_1[u].add(v); adj_cost_1[v].add(u)
    unvisited = set(range(num_nodes))
    zero_components = 0
    for i in range(num_nodes):
        if i in unvisited:
            zero_components += 1
            unvisited.remove(i)
            queue = deque([i])
            while queue:
                u = queue.popleft()
                free_neighbors = unvisited - adj_cost_1[u]
                for v in free_neighbors:

```

```

        unvisited.remove(v); queue.append(v)
    return zero_components - 1

```

6. 其他图论算法

```

# 关键路径 (AOE 网)
from collections import deque
def find_critical_path(n, edges):
    adj = [[] for _ in range(n)]; in_degree = [0] * n
    for u, v, w in edges:
        adj[u].append((v, w)); in_degree[v] += 1
    ve, topo_order = [0] * n, []
    q = deque([i for i in range(n) if in_degree[i] == 0])
    while q:
        u = q.popleft(); topo_order.append(u)
        for v, w in adj[u]:
            in_degree[v] -= 1
            if ve[u] + w > ve[v]: ve[v] = ve[u] + w
            if in_degree[v] == 0: q.append(v)
    if len(topo_order) < n: return -1, []
    max_len = max(ve); vl = [max_len] * n
    for u in reversed(topo_order):
        for v, w in adj[u]:
            if vl[v] - w < vl[u]: vl[u] = vl[v] - w
    critical_edges = []
    for u in range(n):
        for v, w in adj[u]:
            if ve[u] == vl[v] - w: critical_edges.append((u, v, w))
    return max_len, critical_edges

# 割点与割边 (Tarjan)
def find_cut_vertices_and_bridges(num_nodes: int, edges: list[list[int]]):
    graph = {i: [] for i in range(num_nodes)}
    for u, v in edges: graph[u].append(v); graph[v].append(u)
    dfn, low, timer = [-1] * num_nodes, [-1] * num_nodes, 0
    cut_vertices, bridges = set(), []
    def dfs(u, parent):
        nonlocal timer
        dfn[u] = low[u] = timer; timer += 1
        children_count = 0
        for v in graph[u]:
            if v == parent: continue
            if dfn[v] == -1:
                children_count += 1
                dfs(v, u); low[u] = min(low[u], low[v])
                if low[v] > dfn[u]: bridges.append((u, v))
                if parent != -1 and low[v] >= dfn[u]: cut_vertices.add(u)
            else: low[u] = min(low[u], dfn[v])
        if parent == -1 and children_count >= 2: cut_vertices.add(u)
    for i in range(num_nodes):
        if dfn[i] == -1: dfs(i, -1)

```

```

        return list(cut_vertices), bridges

# 欧拉路径
import sys
sys.setrecursionlimit(200005)
def get_eulerian_path(n, edges):
    adj, in_deg, out_deg = [[] for _ in range(n + 1)], [0] * (n + 1), [0] * (n + 1)
    for u, v in edges:
        adj[u].append(v); out_deg[u] += 1; in_deg[v] += 1
    for i in range(1, n + 1): adj[i].sort(reverse=True)
    start_node, start_cnt, end_cnt = 1, 0, 0
    for i in range(1, n + 1):
        if out_deg[i] - in_deg[i] == 1: start_node = i; start_cnt += 1
        elif in_deg[i] - out_deg[i] == 1: end_cnt += 1
        elif in_deg[i] != out_deg[i]: return []
    if not ((start_cnt == 0 and end_cnt == 0) or (start_cnt == 1 and end_cnt == 1)): return []
    path = []
    def dfs(u):
        while adj[u]:
            v = adj[u].pop(); dfs(v)
        path.append(u)
    dfs(start_node)
    if len(path) != len(edges) + 1: return []
    return path[::-1]

```

abc 字符串算法

```

# KMP 算法
def kmp(s, p):
    n, m = len(s), len(p)
    if m == 0: return []
    nxt = [0] * m
    j = 0
    for i in range(1, m):
        while j > 0 and p[i] != p[j]: j = nxt[j - 1]
        if p[i] == p[j]: j += 1
        nxt[i] = j
    res, j = [], 0
    for i in range(n):
        while j > 0 and s[i] != p[j]: j = nxt[j - 1]
        if s[i] == p[j]: j += 1
        if j == m:
            res.append(i - m + 1)
            j = nxt[j - 1]
    return res

# Manacher (马拉车算法)
def manacher(s):

```

```
if not s: return 0, ""
t = '^#' + '#'.join(s) + '#$'
p = [0] * len(t)
c = r = 0
for i in range(1, len(t) - 1):
    if i < r: p[i] = min(r - i, p[2 * c - i])
    while t[i + p[i] + 1] == t[i - p[i] - 1]: p[i] += 1
    if i + p[i] > r: c, r = i, i + p[i]
max_len = max(p)
center_index = p.index(max_len)
start = (center_index - max_len) // 2
longest_palindrome = s[start : start + max_len]
return max_len, longest_palindrome
```



杂项算法

```
# 前缀和 (Prefix Sum)
太简单了
```